

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Optimización de recorridos para la recolección de reciclables

Tesis de Licenciatura en Ciencias de la Computación

Ignacio Fernández Oromendia

Directora: Paula Zabala
Codirectora: Isabel Mendez Díaz
Buenos Aires, 2025

OPTIMIZACIÓN DE RECORRIDOS PARA LA RECOLECCIÓN DE RECICLABLES

TODO

Palabras claves: Guerra, Rebelión, Wookie, Jedi, Fuerza, Imperio (no menos de 5).

OPTIMIZING ROUTES FOR RECYCLABLE COLLECTION

In a galaxy far, far away, a psychopathic emperor and his most trusted servant – a former Jedi Knight known as Darth Vader – are ruling a universe with fear. They have built a horrifying weapon known as the Death Star, a giant battle station capable of annihilating a world in less than a second. When the Death Star's master plans are captured by the fledgling Rebel Alliance, Vader starts a pursuit of the ship carrying them. A young dissident Senator, Leia Organa, is aboard the ship & puts the plans into a maintenance robot named R2-D2. Although she is captured, the Death Star plans cannot be found, as R2 & his companion, a tall robot named C-3PO, have escaped to the desert world of Tatooine below. Through a series of mishaps, the robots end up in the hands of a farm boy named Luke Skywalker, who lives with his Uncle Owen & Aunt Beru. Owen & Beru are viciously murdered by the Empire's stormtroopers who are trying to recover the plans, and Luke & the robots meet with former Jedi Knight Obi-Wan Kenobi to try to return the plans to Leia Organa's home, Alderaan. After contracting a pilot named Han Solo & his Wookiee companion Chewbacca, they escape an Imperial blockade. But when they reach Alderaan's coordinates, they find it destroyed - by the Death Star. They soon find themselves caught in a tractor beam & pulled into the Death Star. Although they rescue Leia Organa from the Death Star after a series of narrow escapes, Kenobi becomes one with the Force after being killed by his former pupil - Darth Vader. They reach the Alliance's base on Yavin's fourth moon, but the Imperials are in hot pursuit with the Death Star, and plan to annihilate the Rebel base. The Rebels must quickly find a way to eliminate the Death Star before it destroys them as it did Alderaan (aprox. 200 palabras).

Keywords: War, Rebellion, Wookie, Jedi, The Force, Empire (no menos de 5).

AGRADECIMIENTOS

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce sapien ipsum, aliquet eget convallis at, adipiscing non odio. Donec porttitor tincidunt cursus. In tellus dui, varius sed scelerisque faucibus, sagittis non magna. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Mauris et luctus justo. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Mauris sit amet purus massa, sed sodales justo. Mauris id mi sed orci porttitor dictum. Donec vitae mi non leo consectetur tempus vel et sapien. Curabitur enim quam, sollicitudin id iaculis id, congue euismod diam. Sed in eros nec urna lacinia porttitor ut vitae nulla. Ut mattis, erat et laoreet feugiat, lacus urna hendrerit nisi, at tincidunt dui justo at felis. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Ut iaculis euismod magna et consequat. Mauris eu augue in ipsum elementum dictum. Sed accumsan, velit vel vehicula dignissim, nibh tellus consequat metus, vel fringilla neque dolor in dolor. Aliquam ac justo ut lectus iaculis pharetra vitae sed turpis. Aliquam pulvinar lorem vel ipsum auctor et hendrerit nisl molestie. Donec id felis nec ante placerat vehicula. Sed lacus risus, aliquet vel facilisis eu, placerat vitae augue.

A mi persona favorita.

Índice general

1.. Introducción 1
1.1. Contexto 1
1.2. Definiciones 1
1.3. Problemas de recorrido 2
1.3.1. Problema del Cartero Chino 2
1.3.2. Problema del Cartero Chino Rural 2
1.3.3. Capacitated Arc Routing Problem 3

2.. Optimizando tiempo 5
2.1. Modelo Inicial 5
2.2. Evitando movimientos prohibidos 6
2.2.1. Súper-Grafo 6
2.3. Fix & Optimize 7
2.3.1. Selección de vecindarios 7
2.4. Zonificación 8

3.. Construcción de Instancias 11
3.1. Sistemas de Información Geográfica 11
3.2. Transformando Mapas en Grafos 11
3.2.1. Zonificando con Grillas 13

4.. Construcción de Soluciones 15
4.1. Transformando Grafos en Recorridos 15

5.. Experimentos y Resultados 17

6.. Conclusiones y próximos pasos 19

1. INTRODUCCIÓN

1.1. Contexto

La provincia de Buenos Aires es una de las zonas más pobladas de la Argentina, con una significativa producción de residuos urbanos. Esto hace que la gestión de residuos reciclables represente un desafío crucial para los municipios, que buscan implementar sistemas eficientes y sostenibles para el reciclaje.

Mediante programas de educación y concientización se ha logrado fomentar la separación en origen, donde los ciudadanos separan los residuos reciclables de los no reciclables en sus hogares. Para complementar esta tarea, algunos municipios de la provincia han implementado la recolección de residuos reciclables por medio de sistemas puerta a puerta, mediante camiones que recorren sus calles de forma semanal, un día específico según la zona del municipio. Estos camiones recogen los residuos de las puertas de los hogares y los llevan a una planta de clasificación, donde los materiales reciclables se separan y procesan.

En el caso de San Isidro, el municipio decidió separar su terreno en 6 zonas distintas siguiendo la idea de las separación barrial. Estas zonas son: Acassuso, Beccar, Boulogne, Martínez, San Isidro y Villa Adelina. Para no alterar la rutina de los vecinos no se van a modificar estas zonas, por lo cual vamos a resolver el problema de optimización para cada una de ellas por separado.

El servicio consiste en que uno o varios camiones recorren las calles de cada zona el día determinado por el municipio, recogiendo las bolsas dejadas por los vecinos. Cuando la capacidad del camión es alcanzada, la recolección realizada hasta ese momento es llevaba a una planta de clasificación. Si fuera necesario, los camiones realizan más de un viaje diario.

El desarrollo realizado por el Departamento de Computación de la Facultad de Ciencias Exactas de la Universidad de Buenos Aires tuvo como objetivo encontrar diferentes alternativas de recorridos que contemplan diversos objetivos y restricciones.

1.2. Definiciones

Un *grafo* G consiste en un par $G = (V, E)$ donde V es un conjunto vértices y E es un conjunto de pares (v, u) las cuales llamaremos aristas. Estos pares son bidireccionales, es decir, podemos ir desde v a u y viceversa. Por otro lado, tenemos los *grafos dirigidos* D, los cuales también consisten en un par $D = (V, E)$ que representan lo mismo que en un grafo salvo que en este caso, los pares (v, u) de E no son bidireccionales. Estos tienen la dirección igual a la de su par, podemos ir de v a u pero no podemos hacer la inversa.

En este problema vamos a trabajar con un *grafo mixto* G el cual contiene ambos tipos de ejes. Es decir, $G = (V, E \cup A)$ donde E contiene las aristas bidireccionales y A las aristas unidireccionales, las cuales llamaremos arcos.

1.3. Problemas de recorrido

No se puede poner una fecha de origen [CAMBIAR ORIGEN] para los problemas de recorrido, pero si podemos decir que fueron el punta pie para el origen de la teoría de grafos. El ejemplo clásico es el problema de los puentes de **Konigsberg** [BUSCAR COMO HACER EL : EN LA O], el cual tiene el nombre de la ciudad a la cual es origen. En esta ciudad tenemos un río que se bifurca y rodea un terreno separando en 4 regiones la ciudad conectada por 7 puentes. El problema consistía en encontrar un recorrido para cruzar a pie toda la ciudad pasando solo una vez por cada uno de los puentes y regresando al mismo punto de inicio. En la actualidad, a este tipo de problemas se lo conoce como circuito euleriano. En el área de optimización combinatoria a este problema se lo conoce como el problema de *cartero chino*

1.3.1. Problema del Cartero Chino

Dada una ciudad con calles e intersecciones, el problema consiste en encontrar el circuito más corto que visite cada calle volviendo al punto de partida. Formalizando el problema del Cartero Chio (CPP), podemos decir que dado un grafo $G = (V, E)$ y una función de costos sobre cada una de las aristas de E queremos determinar un circuito que contenga a todo elemento de E y que la suma de costos de las aristas utilizadas sea mínimo.

Un aspecto curioso sobre este problema es que sus versiones dirigidas y no dirigida tienen una complejidad temporal polinomial, es decir, se encuentra en la clase P de problemas, es decir, es posible resolverlo en tiempo razonable y se considera de una baja dificultad. Con lo cual, uno pensaría que su versión mixta también será accesible de resolver. Sin embargo, todo lo contrario, el problema del cartero chino mixto se encuentra dentro de la clase de complejidad NP-Completo, con lo cual, no existe algoritmo que en complejidad temporal polinomial lo resuelva.

No obstante, podemos modelar este tipo de problemas con programación lineal donde tendremos una función objetivo sujeta a restricciones que nos darán la solución óptima al problema. Para resolver este tipo de problema se usa el algoritmo *SIMPLEX*, desarrollado por George Dantzig en 1947, que a traves de analizar los vértices del polígono generado por el modelo y varias iteraciones de procesamiento y reducción del espacio de búsqueda encuentra la solución óptima. *SIMPLEX* tiene una complejidad teórica temporal exponencial, con lo cual nos deja parados fuera de la clase P. Sin embargo, en la práctica este alogritmo es polinomial, con lo cual, resulta muy útil para resolver problemas de recorrido.

1.3.2. Problema del Cartero Chino Rural

El problema del Cartero Chino Rural (RPP) nos presenta la misma situación que el su versión original. La diferencia se encuentra en que, dado un grafo mixto $G = (V, E \cup A)$ en este caso vamos a tener un conjunto $R \subseteq E \cup A$ de ejes obligatorios a recorrer. Aunque cambia la dificultad del problema ya que no hace falta recorrer todas los ejes si no que sólo los que contiene R, este se encuentra en la clase NP-Completo.

1.3.3. Capacitated Arc Routing Problem

Los problemas CARP se originan a partir del problema del Cartero Chino Rural donde se le agregan al problema una función de capacidad, la cual asigna una demanda a cada eje del grafo, y una capacidad al vehículo que la recorre. Esto nos obliga a tener que partir en distintas rutas en caso de que no se pueda recorrer todos los ejes obligatorios cumpliendo con la restricción de capacidad.

2. OPTIMIZANDO TIEMPO

2.1. Modelo Inicial

Algo que introduzca esto

Dado un grafo $G = (V, E \cup A)$, $R \subseteq E \cup A$ como el conjunto de ejes obligatorios de recorrer, $t : E \cup A \to \mathbb{R}$ como el tiempo en recorrer el eje ij, $q : E \cup A \to \mathbb{R}$ como la demanda del eje ij y W la capacidad de cada camión, definimos las siguientes variables.

$$X_{ij}^p = \begin{cases} 1 & \text{si el eje } ij \text{ fue satisfecho en el viaje } p \\ 0 & \text{caso contrario} \end{cases}$$

$Y_{ij}^p =$ Cantidad de veces que el eje ij fue recorrido sin recoger en el viaje p

$F_{ij}^p =$ Flujo del eje ij en el viaje p

El modelo que en una primera instancia resuelve el problema es el siguiente. Como notación vamos a usar t_{ij} en lugar de $t(ij)$ para referirnos al tiempo en recorrer el eje ij, además usaremos $S = E \cup A$.

$$\text{mín} \quad \sum_{p=1}^{P}\Big(\sum_{ij\in R} t_{ij}X_{ij}^p + \sum_{ij\in S} t_{ij}Y_{ij}^p\Big) \tag{2.1}$$

$$\text{sujeto a} \quad \sum_{j/ij\in S} Y_{ij}^p + \sum_{j/ji\in R} X_{ij}^p = \sum_{j/ji\in S} Y_{ji}^p + \sum_{j/ij\in R} X_{ji}^p \qquad \forall i\ \forall p \tag{2.2}$$

$$\sum_{p=1} X_{ij}^p = 1 \qquad \forall ij \in R \cap A \tag{2.3}$$

$$\sum_{p=1}(X_{ij}^p + X_{ji}^p) = 1 \qquad \forall ij \in R \cap E \tag{2.4}$$

$$\sum_{j:(0j)\in S} Y_{0j}^p + \sum_{j:(0j)\in R} X_{0j}^p \leq 1 \qquad \forall p \tag{2.5}$$

$$\sum_{j:(ji)\in S} F_{ji}^p - \sum_{j:(ij)\in R} F_{ij}^p = \sum_{j:(ji)\in R} q_{ij}X_{ji}^P \qquad \forall i\ \forall p \tag{2.6}$$

$$\sum_{j:(0j)\in S} F_{0j}^p = \sum_{(ij)\in R} q_{ij}X_{ij}^p \qquad \forall i \forall p \tag{2.7}$$

$$\sum_{i:(i0)\in S} F_{i0}^p = \sum_{i:(i0)\in R} q_{i0}X_{i0}^p \qquad \forall p \tag{2.8}$$

$$F_{ij}^p \leq W(y_{ij}^p + x_{ij}^p) \qquad \forall (i,j) \in S, \forall p \tag{2.9}$$

$$X_{ij}^p \in \{0,1\} \qquad \forall (i,j) \in R, \forall p \tag{2.10}$$

$$F_{ij}^p \geq 0 \qquad \forall (i,j) \in S, \forall p \tag{2.11}$$

$$Y_{ij}^p \geq 0 \text{ entera} \qquad \forall (i,j) \in S, \forall p \tag{2.12}$$

La función objetivo (1) del problema representa el costo total del recorrido, es decir, el costo de recoger residuos y el de recorrer por ejes sin recoger. La condición (2) impone continuidad al recorrido sobre cada nodo, que todo nodo tenga la misma cantidad de entradas que salidas. Luego, (3) y (4), obligan a recorrer todos los arcos y aristas respectivamente. En la restricción (5) nos aseguramos de que en cada camión salga del depósito a lo sumo una vez. Luego, las condiciones (6)-(9) tienen la función de conservar el flujo y forzar la conectividad entre cada nodo. Por otro lado, la condición (9) impone una cota súperior que garantiza que la capacidad W no se súpere. Vale aclarar que este modelo, no tiene en consideración al depósito como nodo intermedio.

[Siento que quedó medio vacío]

2.2. Evitando movimientos prohibidos

El modelo inicial resuelve el problema, satisfaciendo tanto la conectividad como la demanda. Sin embargo, no tiene en cuenta ciertas reglas de tránsito que nosotros debemos respetar. Estas reglas son: los **giros en U** y los **giros a la izquierda prohibidos**.

Para resolver ambos problemas debemos contener más información en nuestras intersecciones para poder manejar cada caso. Para ello, vamos a **expandir** cada intersección convirtiendo nuestro grafo en un *súper-grafo*.

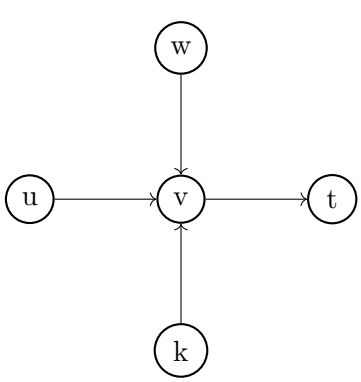
2.2.1. Súper-Grafo

Dado el grafo $G = (V, E \cup A)$ nuestro súper-grafo SG representará cada nodo con un *súper-nodo* y cada arista con una *súper-arista*. El súper-grafo es un grafo dirigido que expande un grafo G, es decir, tantos los nodos como las aristas en el grafo se convierten en conjuntos de nodos y aristas respectivamente.

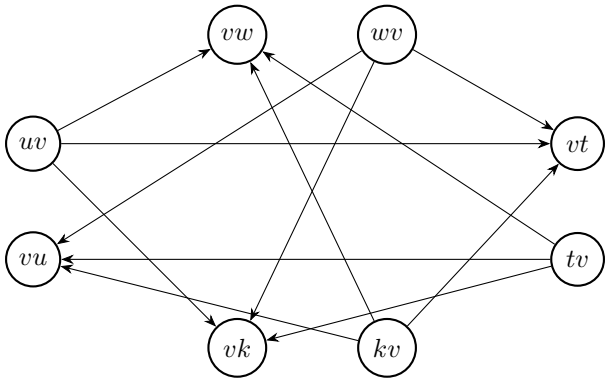
Definamos nuestro súper-grafo $SG = (SV, SE)$, para cada vértice $v \in V$ vamos a representarlo como el conjunto de nodos tal que $uv \in E \cup A$. Es decir, cada nodo $uv \in SV$, indica que nos encontramos en la intersección v viniendo desde la intersección u. Por otro lado, para cada par vértices $uv, vw \in SV$ existe un arco $((uv), (vw))$ si y sólo si existe el camino uvw en G. Formalmente:

- $SV = E \cup A$
- $SA = \{((uv), (kw)) : (uv), (kw) \in SV \wedge v = k \wedge u \neq w\}$

Dado el nuevo modelo con el súper-grafo, podemos fácilmente prohibir los giros en U, ya que no tenemos forma de realizarlo. Luego, para prohibir los giros a la izquierda basta con eliminar todas las aristas tal que representen tal giro y no tengan un semáforo con giro habilitado.



(a) Grafo



(b) Supergrafo

2.3. Fix & Optimize

Esta heurística nace de la motiviación de querer mejorar los tiempos del modelo inical, ya que este puede demandar demasiado tiempo al buscar optimalidad ya que en el espacio de búsqueda es consideradamente grande. Por otro lado, con Fix & Optimize se reduce temporalmente este espacio, obteniendo solcuiones locales con mayor velocidad. Para lograr tal velocidad se debe comenzar con una solcuión facitble del MIP inicial, para luego fijar variables y optimizar las libres.

Es una heurística iterativa donde, mientras haya una mejora, se elige un vecindario, se fijan las variables de recorrido con el valor de la solcuión incumbente salvo las de la zona elegida y se resuelve. Finaliza cuando se llega a un límite de iteraciones sin mejora. El algoritmo itera hasta agotar la cantidad de iteraciones que tiene disponible o si durante 20 iteraciones no mejora.

Lo interesante de esta técnica es como elegir ese vecindario. En esta tesis vamos a analizar 3 maneras de elegir: Aleatorio, Máxima costo de recorridos sin servicio y K-máximas costo de recorridos sin servicio.

2.3.1. Selección de vecindarios

Aleatoria: La selección aleatoria elige una arista unifromemente y selecciona todas las aristas a distancia d.

Máxima costo de recorridos sin servicio: Para este caso se elige la arista con mayor costo de recorridos sin servicio ya que será las que más combinaciones acepte y por ende la que tenga más chances de ser modificada. Para calcular el costo de recorrido sin servicio multiplicamos la cantidad de pasadas sin realizar servicio por el tiempo que toma recorrer dicha arista.

K-máxima costo de recorridos sin servicio: Esta selección realiza un top-k con los parámetros k, d y la tolerancia en el gap. La idea es que si se encuentra una mejora entonces reducimos el k, la cantidad de vecindarios de exploración, la d, el tamaño de los vecindario y mantenemos una tolerancia en el gap más relajada. Sin embargo, si no mejora, alterna entre iteraciones la incrementación de k o d. Si durante 5 iteraciones no se encontró una solución mejor se reduce la tolerancia en el gap.

2.4. Zonificación

A pesar de que el municipio nos dio zonas establecidas siguiendo los barrios, nosotros elegimos zonificar cada una de estas grandes zonas. El motivo de esta zonificación se debe a que el modelo anterior teniendo P camiones tiene muchas posibilidades y es difícil hacer que respete una condición que exige el municipio que es que los camiones se crucen lo menos posible en cuanto a rutas. Por lo tanto, en lugar de resolver el problema con P camiones podemos resolver sub-zonas por separado contemplando un único camión.

Con este nuevo foco sobre el problema tenemos un nuevo problema. Dada una zona, debemos generar P zonas que sean conexas y compactas. Cuando nos referimos a compactas sería que, para 2 zonas se entrelacen lo menos posible entre ellas.

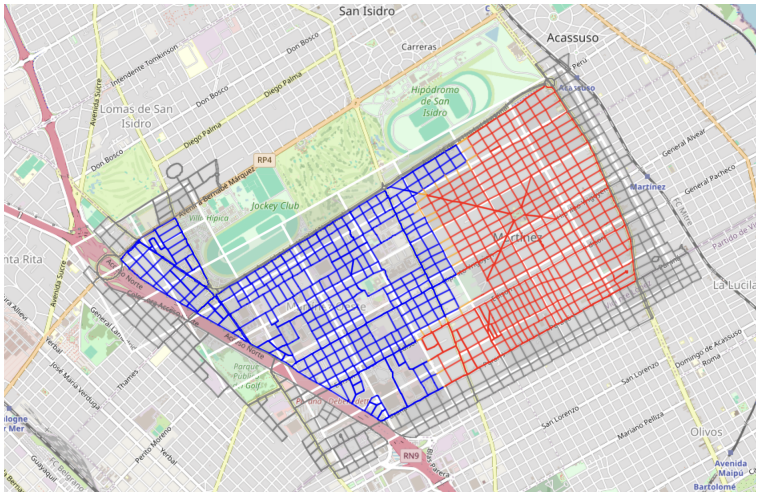


Fig. 2.2: Zonificación

Similar al modelo principal, vamos a pensar a las calles como aristas y las intersecciones como nodos. Primero tenemos que pensar en cual es la función objetivo de este nuevo modelo. Vamos a querer minimizar la distancia máxima entre los nodos de una zona para poder lograr la forma compacta que buscamos. A través de restricciones flujo vamos a lograr que sean zonas conexas y usando otras restricciones auxiliares vamos a asignar cada nodo a una zona.

Dado un grafo $G = (V, E)$, $d : E \to \mathbb{R}$ como la distancia de un eje, $q : V \to \mathbb{R}$ como la demanda de un nodo, U como una cota superior sobre la demanda calculada como $\frac{D_T}{K}$, es decir, la demanda total sobre la cantidad de zonas. W como la capacidad del camión y K como la cantidad de zonas que debemos definir, vamos a precisar de las siguientes variables:

$$X_{ui} = \begin{cases} 1 & \text{si el nodo } u \text{ representa la zona } i \\ 0 & \text{caso contrario} \end{cases}$$

$$Z_{ui} = \begin{cases} 1 & \text{si el nodo } u \text{ es asignado a la zona } i \\ 0 & \text{caso contrario} \end{cases}$$

$$Y_{uv} = \begin{cases} 1 & \text{si los nodos } u \text{ y } v \text{ son asignados a zonas distintas} \\ 0 & \text{caso contrario} \end{cases}$$

$$F_{ij} = \text{Flujo del eje } ij$$

$$D_i = \text{Distancia máxima de la zona } i$$

$$\text{mín} \quad \sum_{i=1}^{K} D_i \tag{2.13}$$

$$\text{sujeto a} \quad \sum_{i=1}^{K} X_{ui} = 1 \tag{2.14}$$

$$X_{ui} - X_{vi} \leq Y_{uv} \qquad \forall i = 1 \dots k \ \forall (u, v) \in E \tag{2.15}$$

$$X_{vi} - X_{ui} \leq Y_{uv} \qquad \forall i = 1 \dots k \ \forall (u, v) \in E \tag{2.16}$$

$$X_{ui} + X_{vi} + Y_{uv} \leq 2 \qquad \forall i = 1 \dots k \ \forall (u, v) \in E \tag{2.17}$$

$$\sum_{u \in V} Z_{ui} = 1 \qquad \forall i = 1 \dots k \tag{2.18}$$

$$Z_{ui} \leq X_{ui} \qquad \forall i = 1 \dots k \ \forall u \in V \tag{2.19}$$

$$F_{uv} + F_{vu} \leq M(1 - Y_{uv}) \qquad \forall (u, v) \in E \tag{2.20}$$

$$\sum_{v \in \delta^-(u)} F_{uv} - \sum_{v \in \delta^+(u)} F_{uv} \geq 1 - M \sum_{i=1}^{K} Z_{ui} \qquad \forall u \in V \tag{2.21}$$

$$D_i \geq d_{uv}(X_{ui} + X_{vi} - 1) \qquad \forall i = 1 \dots k \tag{2.22}$$

$$\sum_{u \in V} X_{ui} q_u \geq 0{,}9W \qquad \forall i = 1 \dots k \tag{2.23}$$

$$\sum_{u \in V} X_{ui} q_u \leq 1{,}05U \qquad \forall i = 1 \dots k \tag{2.24}$$

$$\sum_{i=1}^{K} D_i \geq \frac{K}{1{,}05} D_i \qquad \forall i = 1 \dots k \tag{2.25}$$

$$\sum_{i=1}^{K} D_i \leq \frac{K}{0{,}95} D_i \qquad \forall i = 1 \dots k \tag{2.26}$$

En la restricción (14) forzamos a que un nodo se encuentre únicamente en una zona. En las restricciones (15), (16) y (17) relacionamos las variables X e Y de forma de establecer una restricción de asignación de zona a cada para de nodos. Luego en la (18) elegimos un único representante por zona y en la (19) nos aseguramos de que ese representante efectivamente esté asignado a la zona que representa. En (20) y (21) establecemos el flujo que debe haber entre nodos. En la (22) asignamos la distancia máxima de cada zona. En la (23) y (24) restringimos a la zona contra la capacidad del camión para que sea posible recorrerla recolectando en un único viaje. Además, junto a (25) y (26) hacemos que las K zonas sean balanceadas en demanda y distancia.

3. CONSTRUCCIÓN DE INSTANCIAS

3.1. Sistemas de Información Geográfica

Un *sistema de información geográfica* (SIG) es un conjunto de herramientas que integra y relaciona diversos componentes que permiten la organización, almacenamiento, manipulación y análisis de grandes cantidades de datos procedentes del mundo real que están vinculados a una referencia espacial.

Para obtener la información que nos interesa de un mapa, como calles, coordenadas de esquinas, etc., debemos utilizar un servicio de geolocalización como *Google Maps*. Sin embargo, este no nos proporciona toda la información que necesitamos, por lo tanto, usaremos *Open Street Map (OSM)*, un proyecto colaborativo de origen británico para crear mapas editables y libres.

Dado que las zonas de San Isidro no son rectángulos, más bien son polígonos, tuvimos que generar primero estas figuras utilizando archivos *geojson*, los cuales son archivos *json* con una estructura predefinida específica para contextos geográficos. Por otro lado, no queremos únicamente el polígono de la zona a recorrer, si no que también queremos los alrededores para poder darle la posibilidad al camión de salir de la zona y volver a entrar simulando movimientos permitidos en la realidad. Con lo cual, tendríamos un polígono extendido ligado a una zona, el cual contiene la zona y sus alrededores. Luego a través de una API brindada por *OSM* podemos obtener el mapa con todos los detalles del polígono utilizado.

3.2. Transformando Mapas en Grafos

Una vez generado el mapa *OSM* vamos a querer manipularlo para poder retener únicamente la información que nos importa y agregarle datos adicionales que no se encuentran en *Open Street Maps* que nos brindaron del Municipio. Resumiendo, de cada calle queremos saber si:

- Estamos obligados a recorrerla recolectando residuos.
- Si es mano única o doble.
- Forma parte de un barrio privado o informal.
- Si es una calle de empedrado.
- Si una de sus esquinas es un cruce ferroviario.
- Si una de sus esquinas tiene un semáforo y si este tiene giro a la izquierda.

Para ello vamos a filtrar el mapa *OSM* quedándonos con los elementos que tienen cierto *keyword* identificándolos como una calle para poder generar el grafo mixto G modelando las intersecciones como nodos y las calles como aristas. Por lo tanto, generaríamos un grafo $G = (V, E \cup A)$ donde E es el conjunto de aristas y A el conjunto de arcos.

Sea C el conjunto de calles del mapa *OSM*. El proceso para filtrar el mapa se basa en analizar cada $c \in C$, para ello comenzamos separando en obligatorias o no usando la librearía de python, *shapley*, con la cual podemos verificar si un punto está dentro de un polígono en el mapa. Luego, conociendo las esquinas de la calle c que estamos analizando las vamos a agregar al conjunto de nodos V marcando si tiene alguna característica de interés como cruce de tren, semáforo con o sin luz de giro, etc.

Para realizar esta decisión debemos comparar coordenadas, ya que es la única información en común entre las esquinas del mapa *OSM* y el dataset brindado por el municipio. Luego, dada una lista de puntos A y una esquina candidata e, queremos determinar si existe $a \in A$ tal que e corresponda al mismo punto que a.

Para verificar si dos coordenadas representan la misma esquina utilizamos la distancia euclídea, considerando que $|a - e|_2 < \epsilon$. No buscamos igualdad exacta ya que, al tratarse de coordenadas geográficas, pueden existir pequeñas diferencias en su representación para una misma esquina en distintos mapas o datasets.

Para resolver este pequeño problema vamos a pensar la lista de coordenadas como una matriz A donde cada fila es una coordenada del dataset. A cada fila le vamos a restar el punto e generando una matriz A'.Luego, siendo A'_i la i-ésima fila de A', vamos a corroborar que $\text{mín}_{i=1,\dots,n}(\|A'_i\|_2) < \epsilon$.

Esta operación puede implementarse de manera muy eficiente utilizando *NumPy*, ya que la resta del punto e a todas las filas de la matriz y el cálculo de las normas pueden realizarse mediante operaciones vectorizadas. Estas permiten aplicar una misma operación a todos los elementos de un arreglo de manera simultánea, evitando iteraciones explícitas y aprovechando implementaciones optimizadas de bajo nivel.

Una vez filtrado el mapa vamos a eliminar las calles que no podemos recorrer. Estas calles son las que no tienen salida, ya que el camión no puede dar un giro en U en este tipo de calles. Para ello vamos a eliminar los nodos tal que pertenezcan a alguno de los siguiente 3 conjuntos, una vez eliminados, vamos a volver a generar estos conjuntos y repetir el proceso hasta que al volverlos a generar estén los 3 vacíos.

- $I = \{v \in V : d^+(v) = 0\}$
- $O = \{v \in V : d^-(v) = 0\}$
- $U = \{v \in V : d^+(v) = d^-(v) = 1 \wedge \exists u \in V : uv, vu \in E\}$

El conjunto I representa las intersecciones inaccesible para nosotros, generado en consecuencia de que estamos tomando un polígono el cual deja calles afuera, en particular, calles que conectan puntos fuera del polígono con los que están dentro. Luego, el conjunto O representa intersecciones parecidas a las de I, salvo que son para el otro sentido, es decir, son las que conectan puntos de nuestro polígono con los puntos fuera de el. Finalmente, el conjunto U son las verdaderas calles sin salida que se encuentran en la ciudad.

El siguiente paso es calcular los costos de cada arista o arco, tiempo, dist, demanda...

Otro dato de interés son las curvas, ya que queremos minimizar la cantidad de curvas tomadas por el camión. Para identificar las curvas vamos a utilizar los nombres de cada calle, con lo cual, dadas 3 intersecciones, u, v y w, estando parados en la intersección v,

viniendo de la calle que une las intersecciones de u a v, continuando por la calle que une v y w, vamos a marcar como curva el recorrido uvw si y sólo si los nombres de las calles que unen las intersecciones difieren.

Continuando con las curvas, además de marcarlas, vamos a identificar cuales son curvas a la izquierda y si están prohibidas. Para ello, vamos a analizar las triplas para las cuales la calle que une la intersecciones u y v es doble mano o una avenida sin semáforo para girar a la izquierda. Una vez tenemos el conjunto de candidatas, vamos a calcular el ángulo orientado en sentido horario que generan y si este este pertenece al intervalo $[threshold\pi]$ lo consideraremos como un giro a la izquierda ilegal.

En la siguiente figura podemos visualizar el mapa *OSM* filtrado, donde las calles de color *amarillo* son las calles obligatorias de recorrer, las *azules* son las calles empedradas y las *grises* son las no obligatorias.

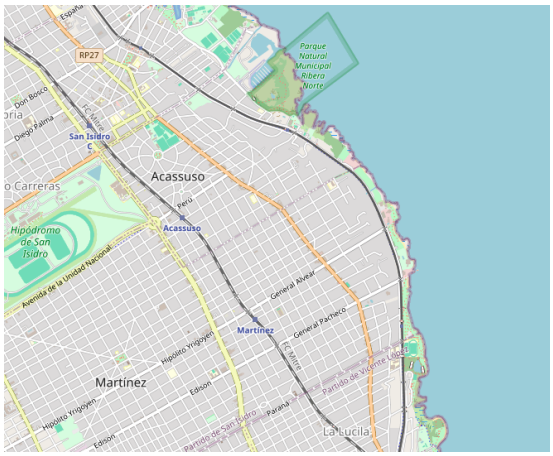


Fig. 3.1: Acassuso *OSM*

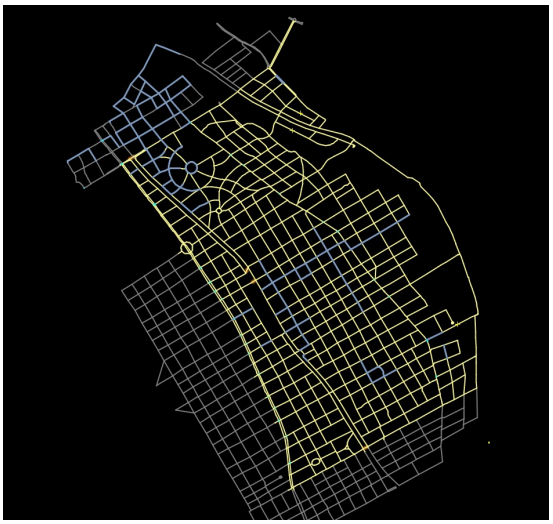


Fig. 3.2: Acassuso *OSM* Filtrado

Recapitulando, a partir de un polígono generamos un una instancia del problema donde tenemos el grafo $G = (V, E \cup A)$, es decir el área a recolectar, y 2 funciones $t : E \cup A \to \mathbb{R}$ y $d : E \cup A \to \mathbb{R}$ que representan, para todo eje perteneciente a G, el costo en tiempo (seg) de recorrerlo y la demanda de basura del mismo respectivamente.

3.2.1. Zonificando con Grillas

Para poder zonificar el grafo en k debemos procesar el mapa como fue explicado anteriormente. En un principio, no tuvimos problemas con el modelo, sin embargo, a medida que lo fuimos probando con instancias reales, el modelo comenzó a demorar mucho más de lo esperado. El objetivo de la zonificación era obtener un resultado concreto y rápido para que luego el modelo de optimización de recorridos encuentre la solución óptima.

Llegamos a la conclusión que estábamos entrando muy en detalle utilizando todos las intersecciones y calles del área para asignarles una zona. Para este problema auxiliar no queríamos una solución óptima de la distribución de calles, necesitábamos una solución eficaz y eficiente. Por lo tanto, decidimos separar el mapa en una grilla, donde cada celda

contendría un conjunto de calles e intersecciones y representaría un nodo en el modelo, reduciendo los nodos un 98 % y las aristas un 93 % en promedio. [CHEQUEAR PORCENTAJES]

El primer paso para generar la grilla es tomar las coordenadas extremas en latitud y longitud. A partir de este paso ya tenemos una grilla que contiene todos los nodos y aristas del grafo. Sin embargo, como el mundo no es perfecto, las calles de nuestra ciudad no están perfectamente alineadas con la latitud o la longitud, con lo cual, vamos a tener que rotar la grilla.

Para poder rotar la grilla necesitamos tener un ángulo de rotación, para ello, necesitamos formar un triangulo en nuestro mapa, donde el hipotenusa quede alineado con una calle seleccionada previamente para alinear la grilla y el cateto 2 paralelo a la longitud. Una vez generado el triángulo basta obtener el ángulo entre el cateto 1 y la hipotenusa para saber cuanto necesitamos girar la grilla. Una vez que tenemos el ángulo vamos a utilizar el método de **Givens** usado para rotar matrices 4×4 para rotar nuestra grilla y alinearla a las calles de la zona.

[DECIR QUE LA DISTANCIA SE CALCULA DE OTRA MANERA EN LA TIERRA?]

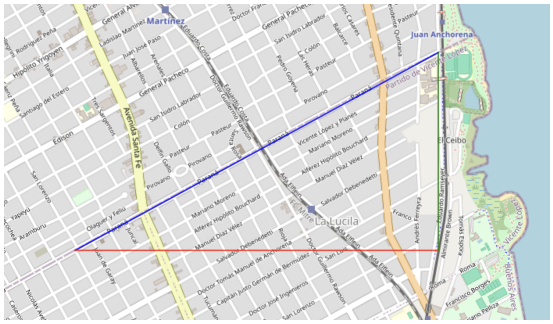


Fig. 3.3: Triángulo de rotación

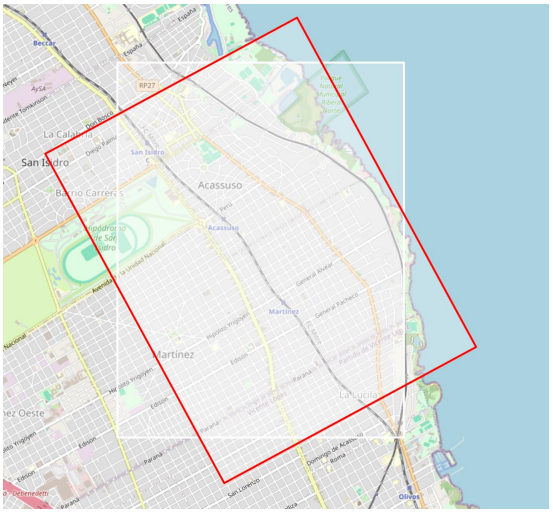


Fig. 3.4: Grilla rotada vs Grilla original

El siguiente paso es generar las celdas, para ello, a través de un parámetro externo sobre la máxima cantidad de celdas, dividimos equitativamente la grilla para obtener lo más parecido a este parámetro. Luego, debemos iterar celda por celda para poder calcular la demanda de cada una de ellas con la siguiente fórmula:

$$D(C) = \sum_{(u,v) \in E \wedge (u,v) \in C} d((u,v))$$

Esto se traduce a decir que la demanda D de cada celda C es la sumatoria de la demanda $d : E \to \mathbb{R}$ de toda arista (u, v) que se encuentra dentro de la celda.

Finalmente obtuvimos una grilla la cual podemos traducir a un grafo grilla con demanda por nodo y distancia entre ellos calculada desde el centro de cada grilla.

4. CONSTRUCCIÓN DE SOLUCIONES

4.1. Transformando Grafos en Recorridos

Una vez finalizado el procesamiento del mejor recorrido obtenemos una asignación de cada calle a un camión en particular. Con lo cual, sabemos que calles recorre ese camión, donde comienza el recorrido y que su recorrido es un circuito, es decir termina donde comienza, en el depósito.

Para armar el recorrido final uno pensaría que basta con concatenar las calles asignadas a un mismo camión. Sin embargo, no cualquier concatenación de calles nos deja con un circuito como el que queremos.

Vamos a querer encontrar circuitos eulerianos en cada región, donde una región son las calles que debe recorrer el camión. Para ello, vamos a ir consumiendo aristas partiendo desde el depósito, donde cada nodo tiene igual grado de entrada que de salida, asegurandonos un circuito. Siempre que el nodo tenga una única entrada y una única posible salida la decisión es trivial, el problema surge cuando ese número es mayor a 1. En este caso, sabemos que va a haber solución pero tenemos que tomar la decisión si continuar por la misma vía o cambiar de vía haciendo que el camión gire en la realidad.

C

5. EXPERIMENTOS Y RESULTADOS

6. CONCLUSIONES Y PRÓXIMOS PASOS